

ABC454 F - Make it Palindrome 2 题解

题意概括

可以对任意子数组执行加一取模操作：

- 选择区间 $[1, r]$
- 把其中每个元素都变成 $(A_i + 1) \bmod M$

问最少做多少次操作，才能让整个序列变成回文。

解题思路

第一步：把目标变成“左右差值”

对于每一对对称位置：

- 左边是 i
- 右边是 $N + 1 - i$

要想最后成为回文，就必须满足：

$$A_i + x_i \equiv A_{N+1-i} + x_{N+1-i} \pmod{M}$$

这里 x_i 表示位置 i 总共被加了多少次。

移项后得到：

$$x_i - x_{N+1-i} \equiv A_{N+1-i} - A_i \pmod{M}$$

设：

$$d_i = (A_{N+1-i} - A_i) \bmod M$$

只需要处理前半位置即可。

第二步：区间操作对应“对一段连续的差值加一或减一”

把每个操作 $[1, r]$ 对这些 d_i 的影响画出来，会发现：

- 如果区间只覆盖了某些对称对的左端点，那么这些 d_i 会整体 +1
- 如果只覆盖了某些对称对的右端点，那么这些 d_i 会整体 -1
- 受影响的 i 总是形成一段连续区间

所以原问题等价于：

- 初始所有 $d_i = 0$
- 每次可以对任意一个连续区间整体 $+1$ 或 -1
- 最终要让第 i 个位置变成与原题要求同余的值
- 问最少多少次操作

第三步：改写成差分

设前一半长度为 $m = \lfloor N/2 \rfloor$ ，并补上：

- $d_0 = 0$
- $d_{m+1} = 0$

再定义差分：

$$r_i = (d_i - d_{i-1}) \bmod M \quad (1 \leq i \leq m+1)$$

如果我们将真正选择的整数差分记成 x_i ，那么它必须满足：

- $x_i \equiv r_i \pmod{M}$
- $x_1 + x_2 + \dots + x_{m+1} = 0$

而最少操作次数恰好等于这些差分中正数部分之和。

第四步：最优选择只会是 r_i 或 $r_i - M$

因为 $0 \leq r_i < M$ ，最优时每个 x_i 只需要在下面两种里选一个：

- r_i
- $r_i - M$

设所有 r_i 的和为 R 。由于 $d_0 = d_{m+1} = 0$ ，所以 R 一定是 M 的倍数。记：

$$K = \frac{R}{M}$$

这表示我们必须恰好选 K 个位置，把它们从 r_i 改成 $r_i - M$ ，这样总和才会变成 0。

一旦某个位置选了 $r_i - M$ ，它就不再贡献正数部分；否则会贡献 r_i 。

因此最优策略就是：

- 选出 K 个最大的 r_i
- 把它们改成 $r_i - M$

答案就是：

R – (最大的 K 个 r_i 之和)

正确性说明

先看等价变形。

一次对原数组的区间加一操作，只会改变那些“左右两个对称位置里恰好被覆盖一个”的对称对，而这样的对称对编号一定构成一个连续区间。因此它在 d 数组上的效果恰好是“对一个连续区间整体加一或减一”。反之，这样的连续区间操作也都能由原数组中的某个区间操作实现。

再看差分。

如果对 d 的某个连续区间整体加一，那么只会让这段区间左端的差分 $+1$ 、右端后一位的差分 -1 。因此每次操作对应把差分数组中的某个正量和某个负量各调节一次，最少操作次数就等于正差分总和。

对于每个差分位置 i ，真正取的整数 x_i 必须满足 $x_i \equiv r_i \pmod{M}$ 。由于 $0 \leq r_i < M$ ：

- 如果取 r_i ，它对答案贡献 r_i
- 如果取 $r_i - M$ ，它不贡献正数部分
- 再多减一个 M 只会让总和更偏负，不会更优
- 取 $r_i + M$ 只会让正数部分更大，也不会更优

所以最优时每个位置只会在 r_i 和 $r_i - M$ 中二选一。

设选了 K 个位置减去 M ，为了让所有差分和为 0 ，必须有：

$$R - KM = 0$$

也就是 $K = R / M$ 。此时答案等于所有未被选中的 r_i 之和，所以显然应当选择 K 个最大的 r_i 去减掉。算法正确。

复杂度

设 $m = \lfloor N/2 \rfloor$ 。

- 时间复杂度： $O(N \log N)$
- 空间复杂度： $O(N)$

参考实现

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
```

```
int main(){

    int T;
    cin >> T;

    while(T--){
        int n;
        ll M;
        cin >> n >> M;

        vector<ll> a(n + 1);
        for(int i=1; i<=n; i++){
            cin >> a[i];
        }

        int m = n / 2;
        vector<ll> d(m + 2, 0), diff;
        diff.reserve(m + 1);

        for(int i=1; i<=m; i++){
            d[i] = (a[n + 1 - i] - a[i] + M) % M;
        }

        ll sum = 0;
        for(int i=1; i<=m + 1; i++){
            ll r = (d[i] - d[i - 1] + M) % M;
            diff.push_back(r);
            sum += r;
        }

        ll need = sum / M;
        sort(diff.begin(), diff.end(), greater<ll>());

        ll cut = 0;
        for(int i=0; i<need; i++){
            cut += diff[i];
        }

        cout << sum - cut << '\n';
    }
}
```

```
return 0;
```

```
}
```

